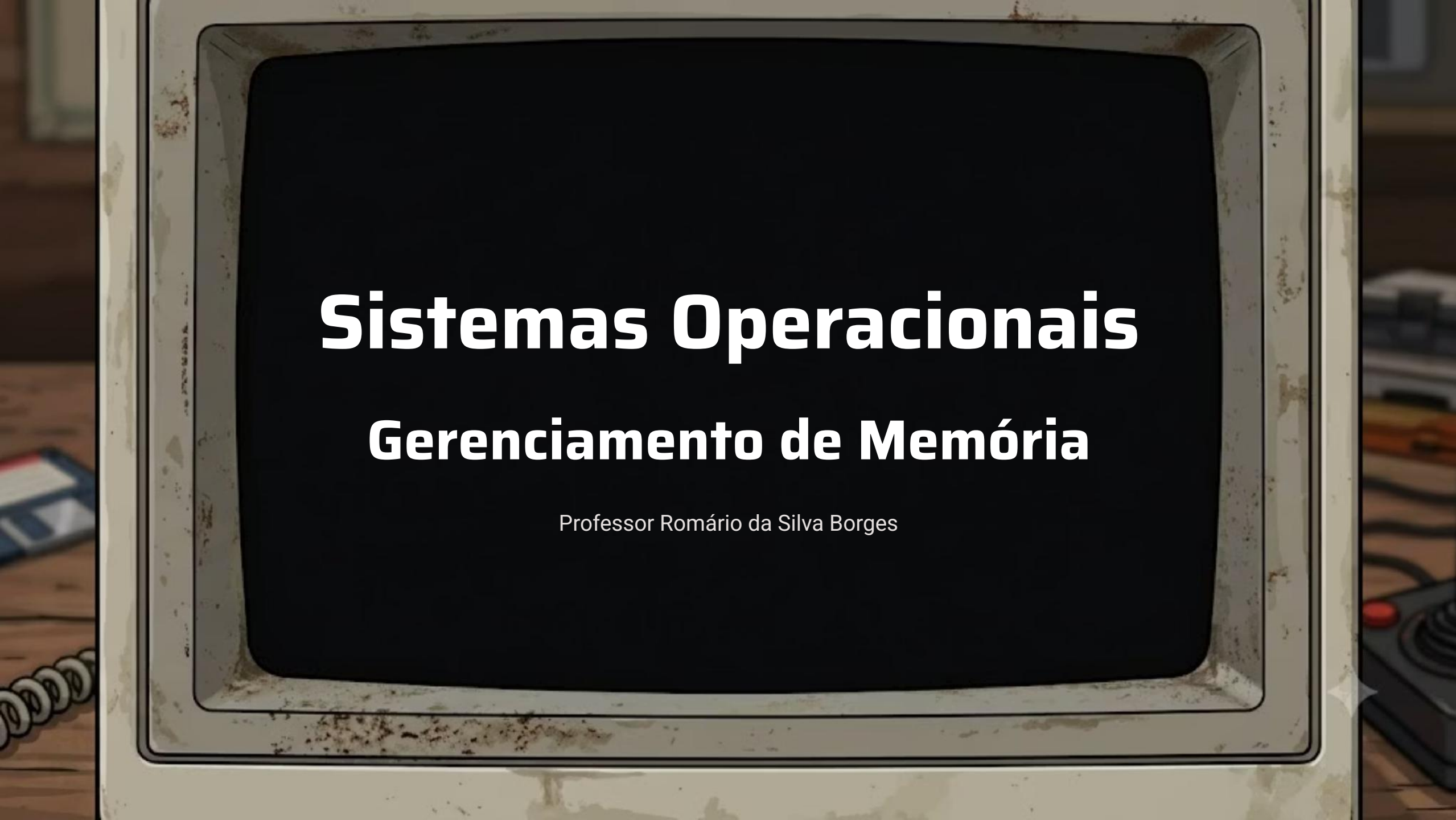




Sistemas Operacionais

Gerenciamento de Memória

Professor Romário da Silva Borges

O que o Gerenciador de Memória faz?

Alocar memória

Para novos processos que chegam ao sistema

Liberar memória

De processos finalizados

Expandir memória

Quando um processo precisa de mais espaço

Controlar endereçamento

Gerencia espaços de endereçamento dos processos

Realizar swapping

Quando necessário, troca processos com o disco

Hardware de Memória

O desejo do programador

Expectativa:

Infinitamente grande

Sem preocupação com limites de espaço

Extremamente rápida

Sem atrasos no acesso aos dados

Não volátil

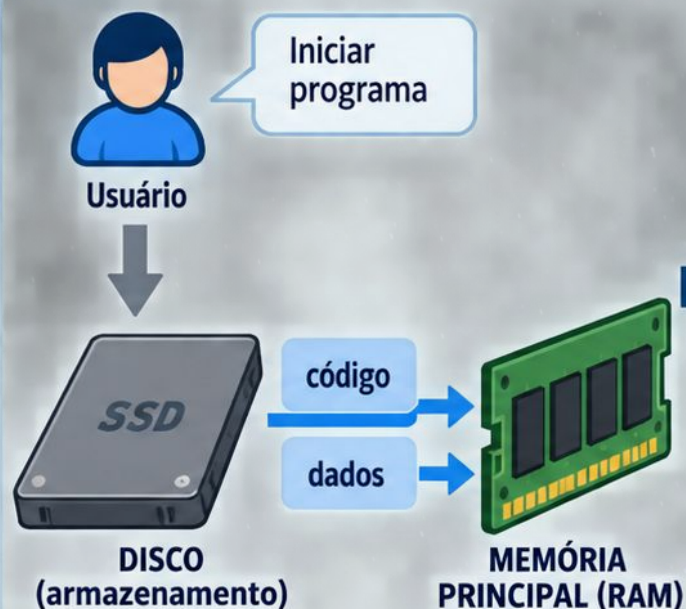
Dados preservados mesmo sem energia

Realidade:

“Programas tendem a expandir-se a fim de preencher a memória disponível para contê-los” - A. Tenenbaum

COMO UM PROGRAMA É EXECUTADO

1 CARREGAMENTO



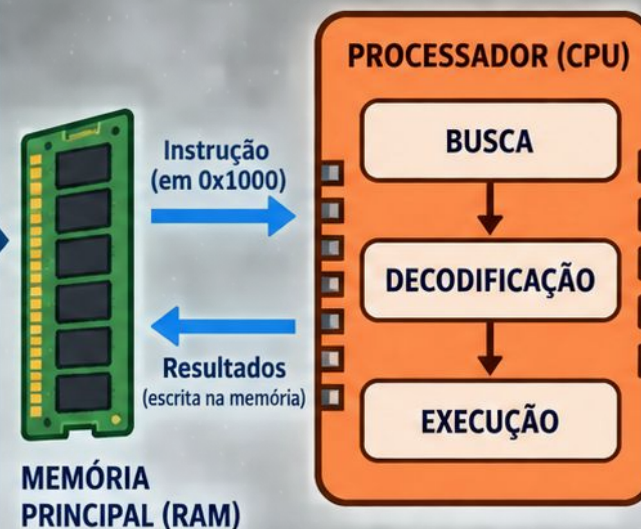
1. O código e os dados do programa são carregados do disco para a memória principal.

2 ENDEREÇAMENTO



2. O PC aponta para o endereço da próxima instrução na memória.

3 EXECUÇÃO



3. O processador busca na memória e executa as instruções do programa.

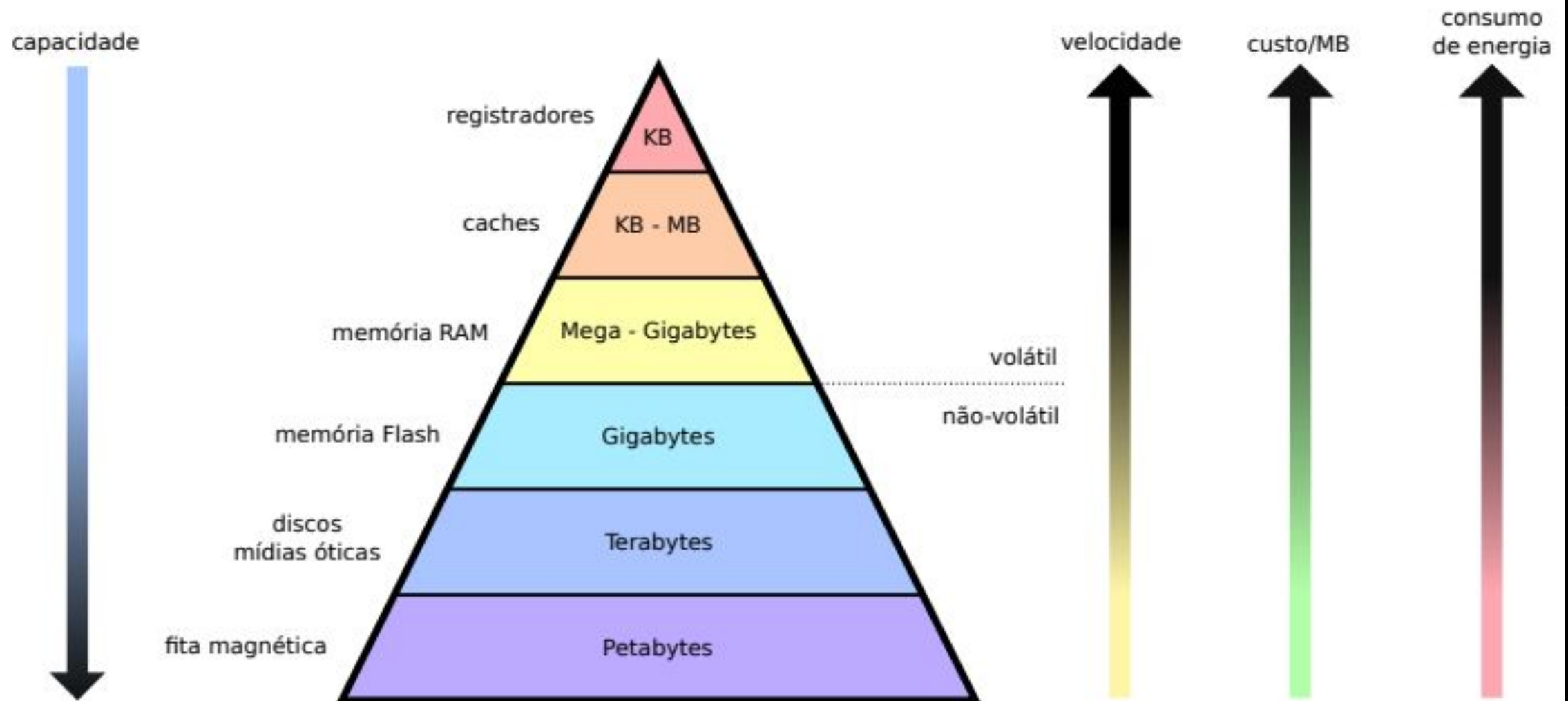


Figura 14.1: Hierarquia de memória.

Meio	Tempo de acesso	Taxa de transferência
Cache L2	1 ns	1 GB/s (1 ns por byte)
Memória RAM	60 ns	1 GB/s (1 ns por byte)
Memória <i>flash</i> (NAND)	2 ms	10 MB/s (100 ns por byte)
Disco rígido SATA	5 ms (tempo para o ajuste da cabeça de leitura e a rotação do disco até o setor desejado)	100 MB/s (10 ns por byte)
DVD-ROM	de 100 ms a vários minutos (caso a gaveta do leitor esteja aberta ou o disco não esteja no leitor)	10 MB/s (100 ns por byte)

Tabela 14.1: Tempos de acesso e taxas de transferência típicas [Patterson and Henessy, 2005].

Memória física

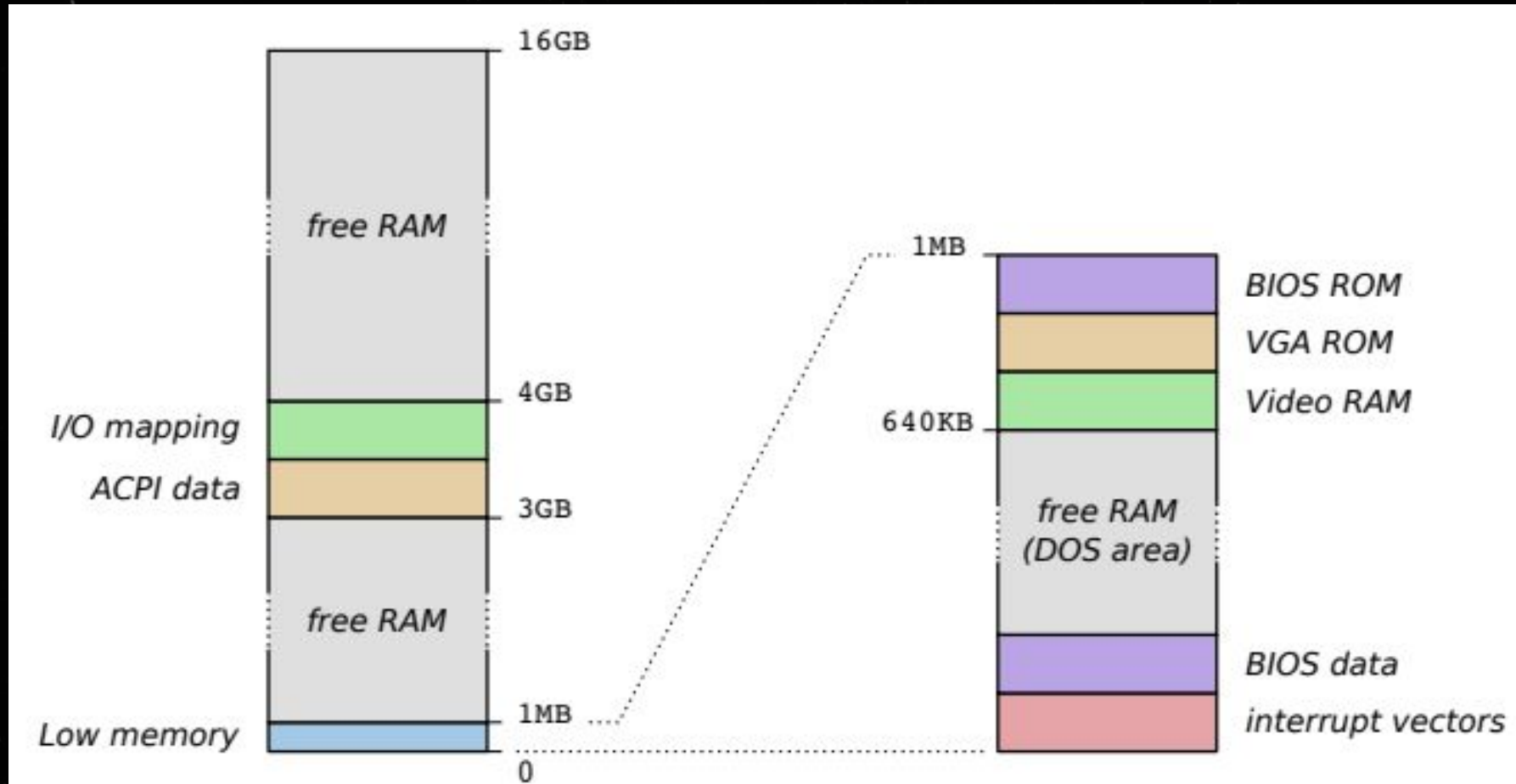


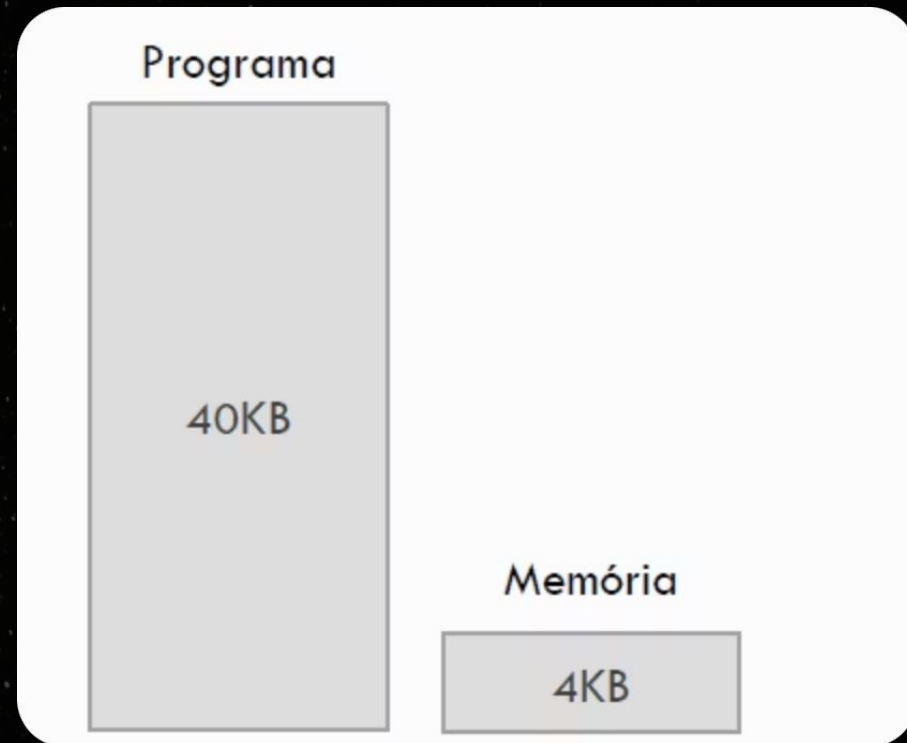
Figura 14.2: Layout da memória física de um computador.

Memória virtual

O Problema do espaço

Desde os primórdios da computação, existem programas maiores que a memória disponível.

Como executar algo que não cabe inteiro na memória?



Solução inicial: Overlays

40KB

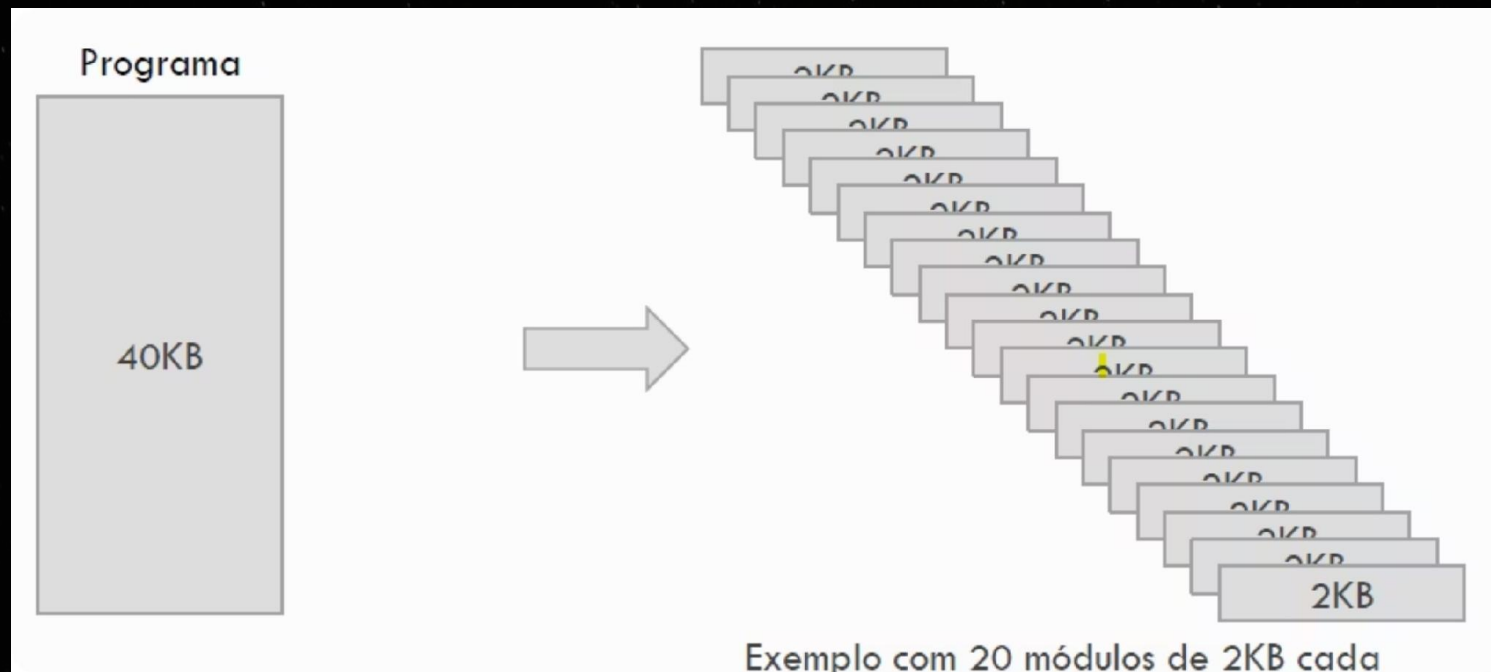
20

2KB

Tamanho do programa

Módulos de 2 KB cada

Por módulo na memória



Funcionamento dos overlays

1 - Módulo inicial

2 - Carregamento on-demand

3 - Coordenação pelo programador

A troca de módulos ocorria **durante a execução** — o programador precisava organizar essa interação manualmente.

Da sobreposição à memória virtual

A comunidade científica percebeu que a responsabilidade de gerenciar partes do programa poderia ser **transferida ao sistema operacional**.

1961

Surge o conceito de **memória virtual**

O SO escolhe

Quais partes do programa ficam na memória

Carga dinâmica

Partes do programa são carregadas conforme necessário

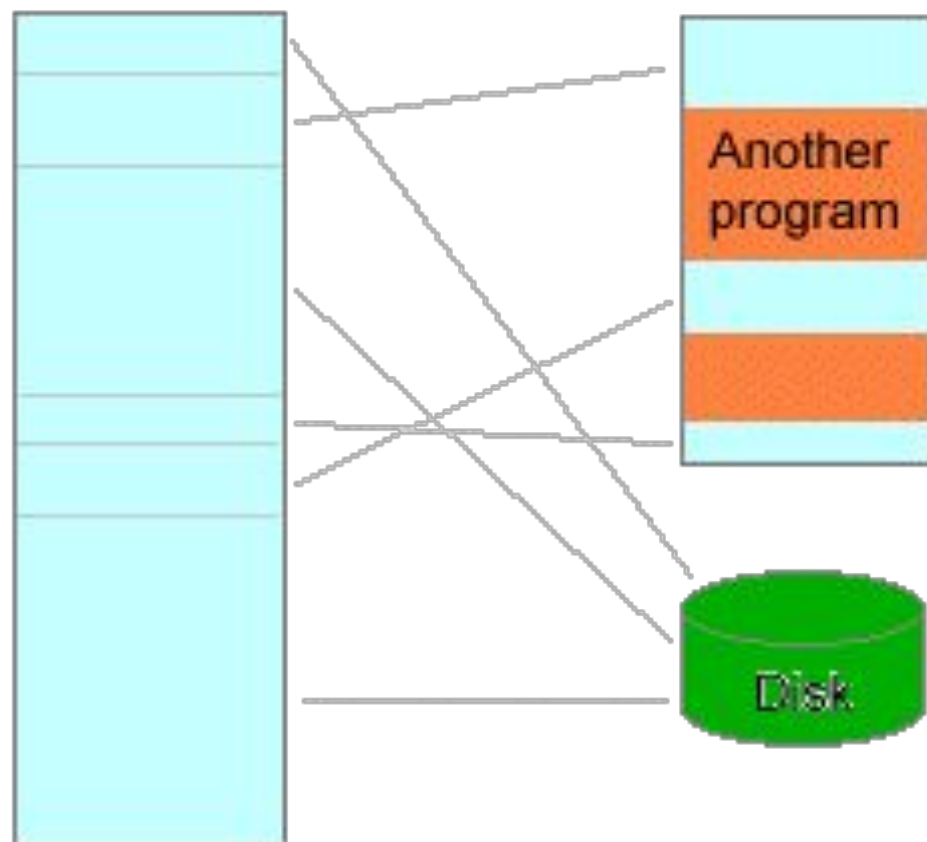
Programa maior

Pode ser executado mesmo sem caber inteiro na RAM

A **Memória virtual** cria a ilusão de que o processo possui mais memória do que a RAM física disponível.

Application sees:

But in reality:



Endereço físico e lógico

Endereços físicos: correspondem às posições dos bytes na memória física do computador. A quantidade desses endereços depende da capacidade de memória disponível na máquina.

Endereços lógicos: também chamados de virtuais, são utilizados pelos processos, pelo sistema operacional e pelo processador durante a execução dos programas. Eles são definidos de acordo com o espaço de endereçamento suportado pelo processador.

MMU (*Unidade de Gerenciamento de Memória*)

A tradução de endereços lógicos em físicos é feita por um componente específico do hardware do computador, denominado Unidade de Gerência de Memória (MMU – Memory Management Unit). Na maioria dos processadores atuais, a MMU se encontra integrada ao chip da própria CPU.

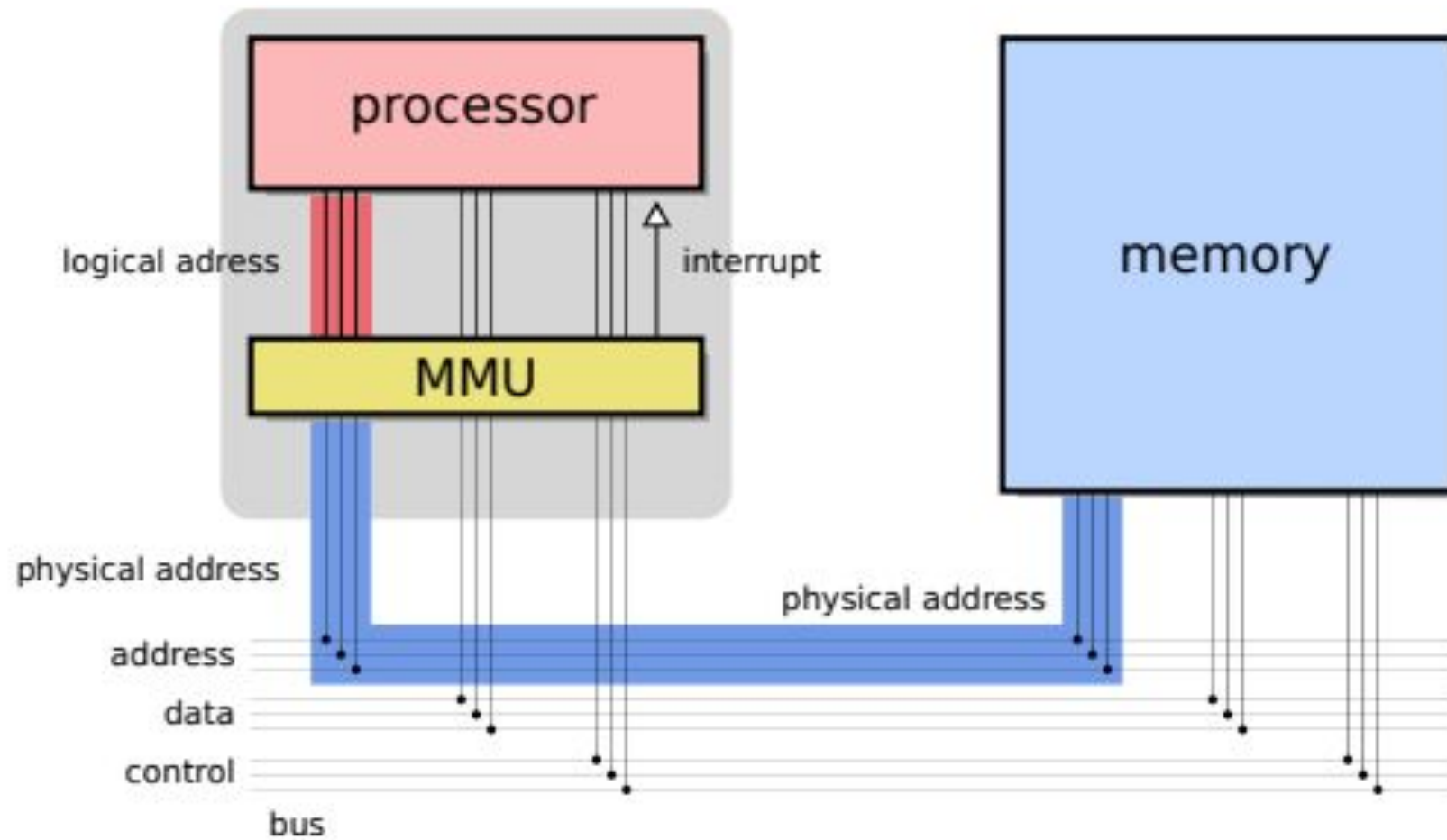


Figura 14.3: Funcionamento básico de uma MMU.

Memória virtual por partições

Memória virtual por partições

A memória física pode ser dividida em várias partições, de tamanhos iguais ou diferentes, fixos ou variáveis. Cada partição armazena um processo. Para converter um endereço lógico em endereço físico, a MMU utiliza dois registradores:

Registrador-base (B): contém o endereço físico inicial da partição ocupada pelo processo.

Registrador-limite (L): contém o tamanho da partição em bytes.

Quando o processador gera um endereço lógico, a MMU verifica se ele está dentro dos limites da partição.

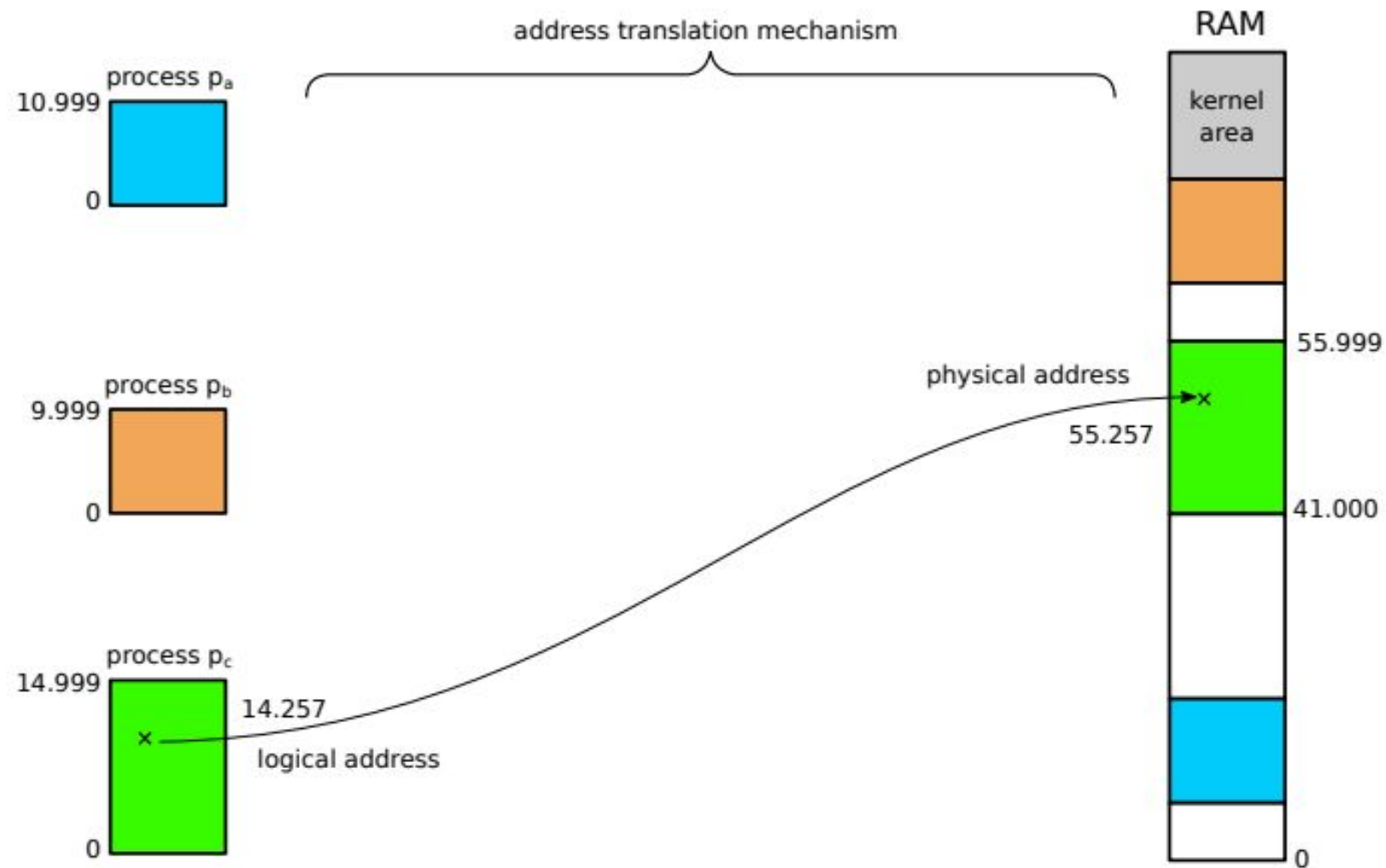


Figura 14.4: Memória virtual por partições.

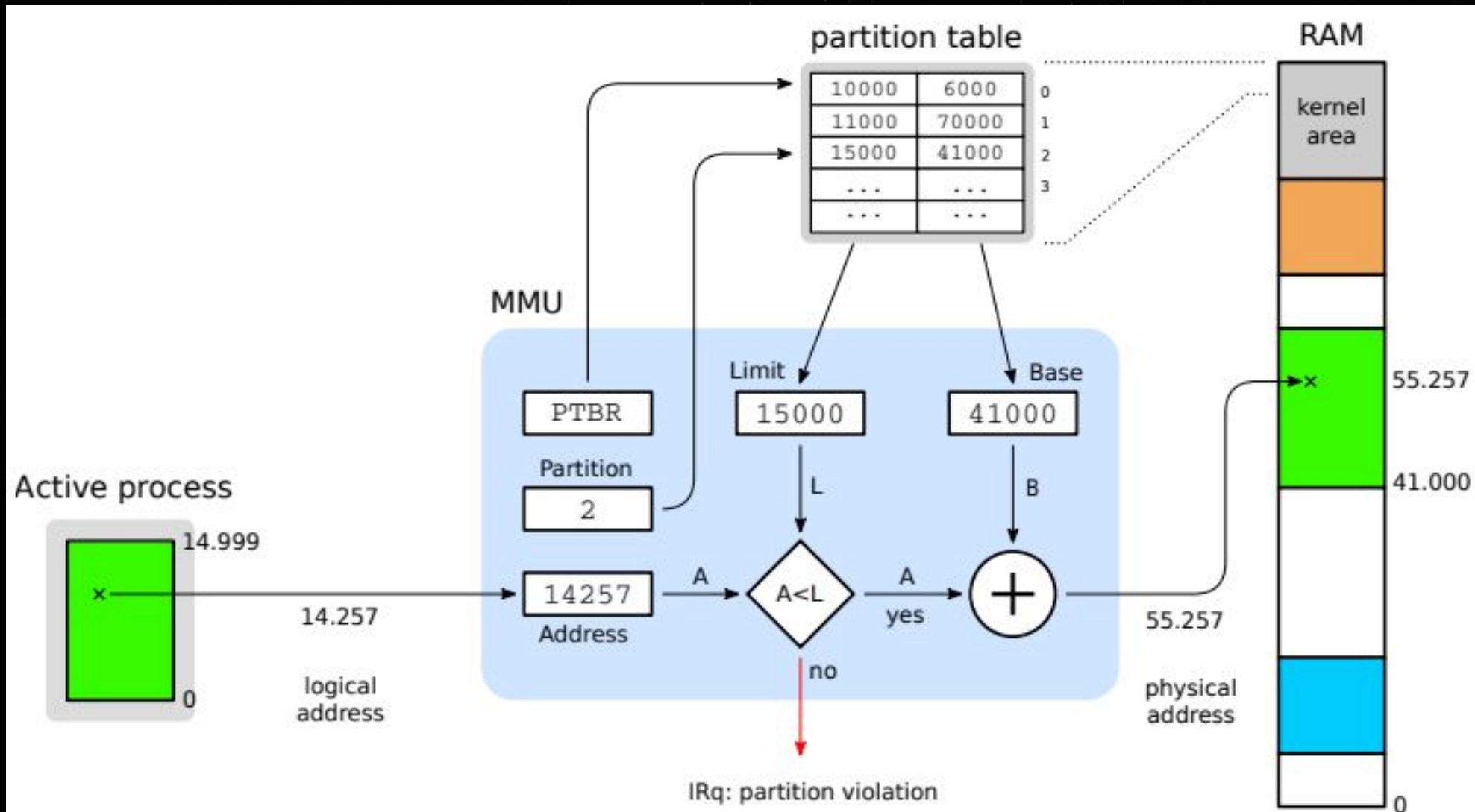


Figura 14.5: MMU com partições.

Memória virtual por segmentos

Memória virtual por segmentos

A segmentação é uma extensão do esquema de partições. Nela, as diferentes seções de um processo, como **TEXT**, **DATA**, **HEAP** e **STACK**, são armazenadas separadamente na memória física. Dessa forma, o espaço de endereçamento do processo deixa de ser uma única sequência linear e **passa a ser organizado como um conjunto de segmentos**. Cada segmento pode possuir tamanho, localização e permissões de acesso diferentes.

Os endereços lógicos gerados pelo processo são representados pelo par:
[segmento:offset]

Segmento: identifica a área de memória que será acessada.

Offset: indica a posição desejada dentro desse segmento.

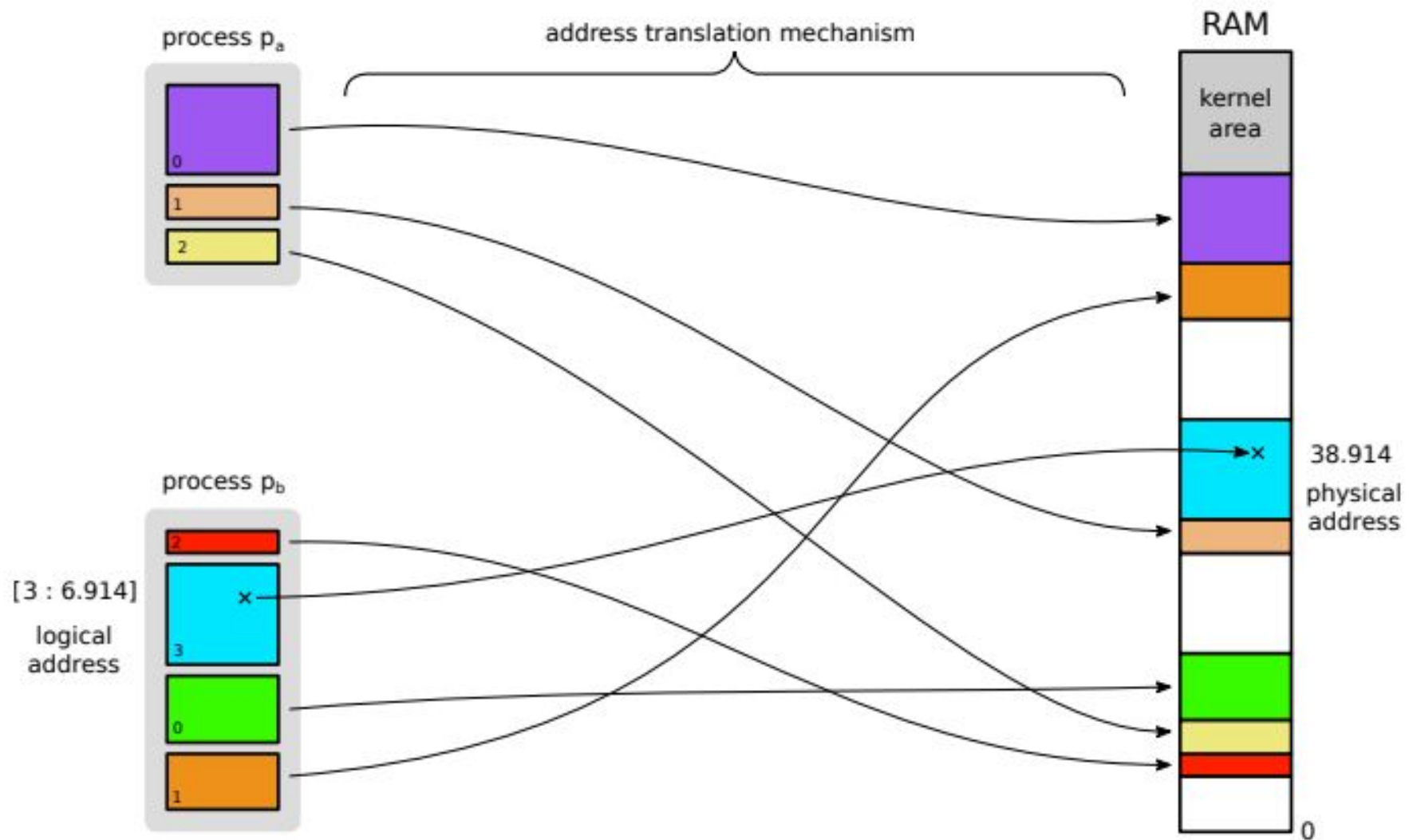


Figura 14.6: Memória virtual por segmentos.

Memória virtual por páginas

Memória virtual por páginas

Na organização da memória por paginação, o espaço de endereçamento lógico de cada processo permanece linear e unidimensional. Entretanto, internamente e de maneira transparente para o processador, esse espaço é dividido em blocos de tamanho fixo chamados páginas.

A memória física também é dividida em blocos do mesmo tamanho, denominados quadros de **página** ou frames - as páginas geralmente têm 4 KBytes (4.096 bytes). **As páginas de um processo podem ser armazenadas em diferentes quadros, sem a necessidade de ocupar posições consecutivas na memória física.**

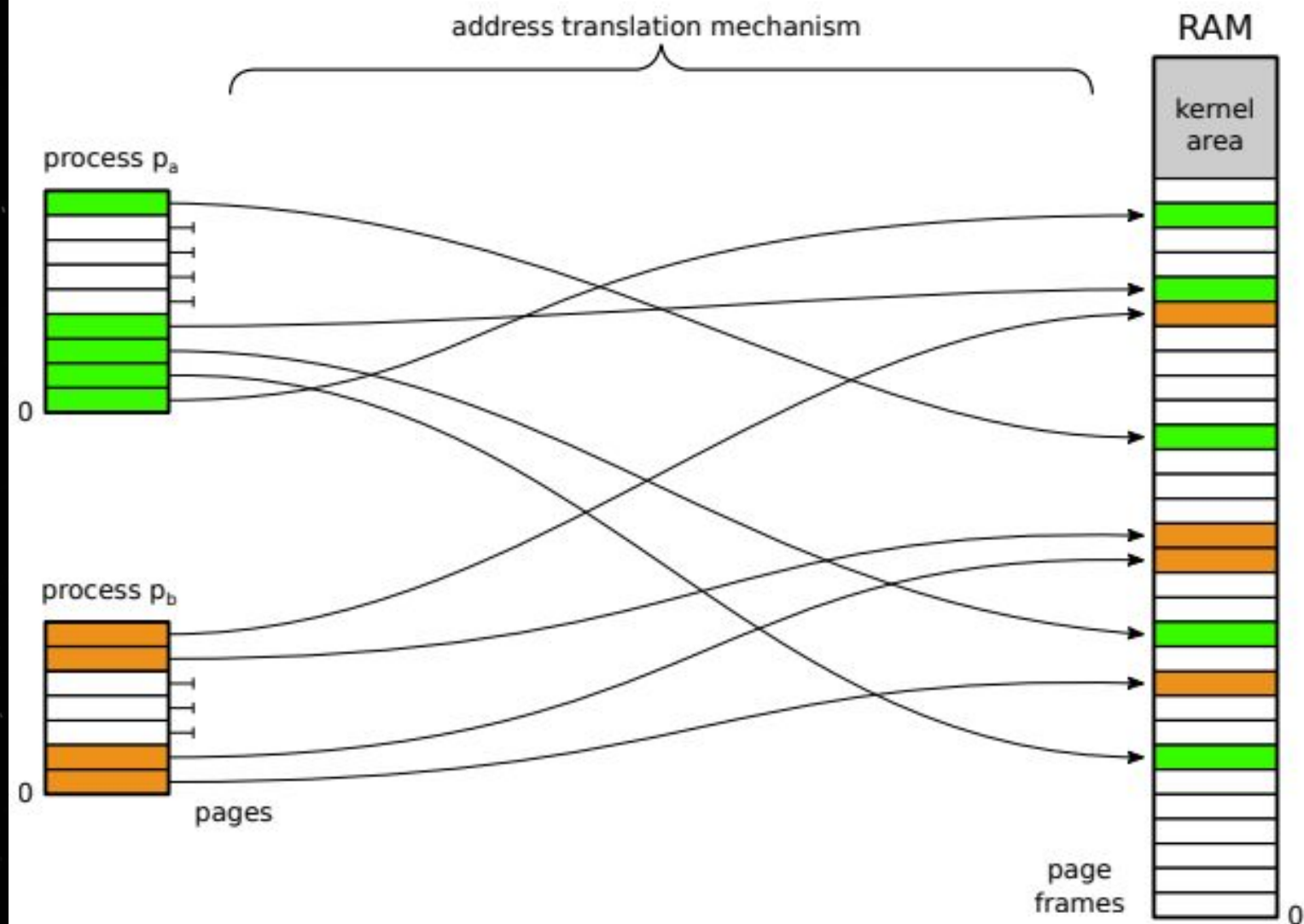


Figura 14.8: Organização da memória em páginas.

Tabela de páginas

O mapeamento entre as páginas e os quadros correspondentes na memória física é feita através de tabelas de páginas (*page tables*), nas quais cada entrada corresponde a uma página do processo e contém o número do quadro onde ela se encontra.

Cada processo possui sua própria tabela de páginas; a tabela de páginas ativa, que corresponde ao processo em execução no momento, é referenciada por um registrador da MMU denominado PTBR – Page Table Base Register. A cada troca de contexto, esse registrador deve ser atualizado com o endereço da tabela de páginas do novo processo ativo.

Para traduzir um endereço lógico no endereço físico correspondente, a MMU efetua os seguintes passos, que são ilustrados na Figura 14.9:

1. decompor o endereço lógico em número de página e *offset*;
2. obter o número do quadro onde se encontra a página desejada;
3. construir o endereço físico, compondo o número do quadro com o *offset*; como páginas e quadros têm o mesmo tamanho, o valor do *offset* é preservado na conversão.

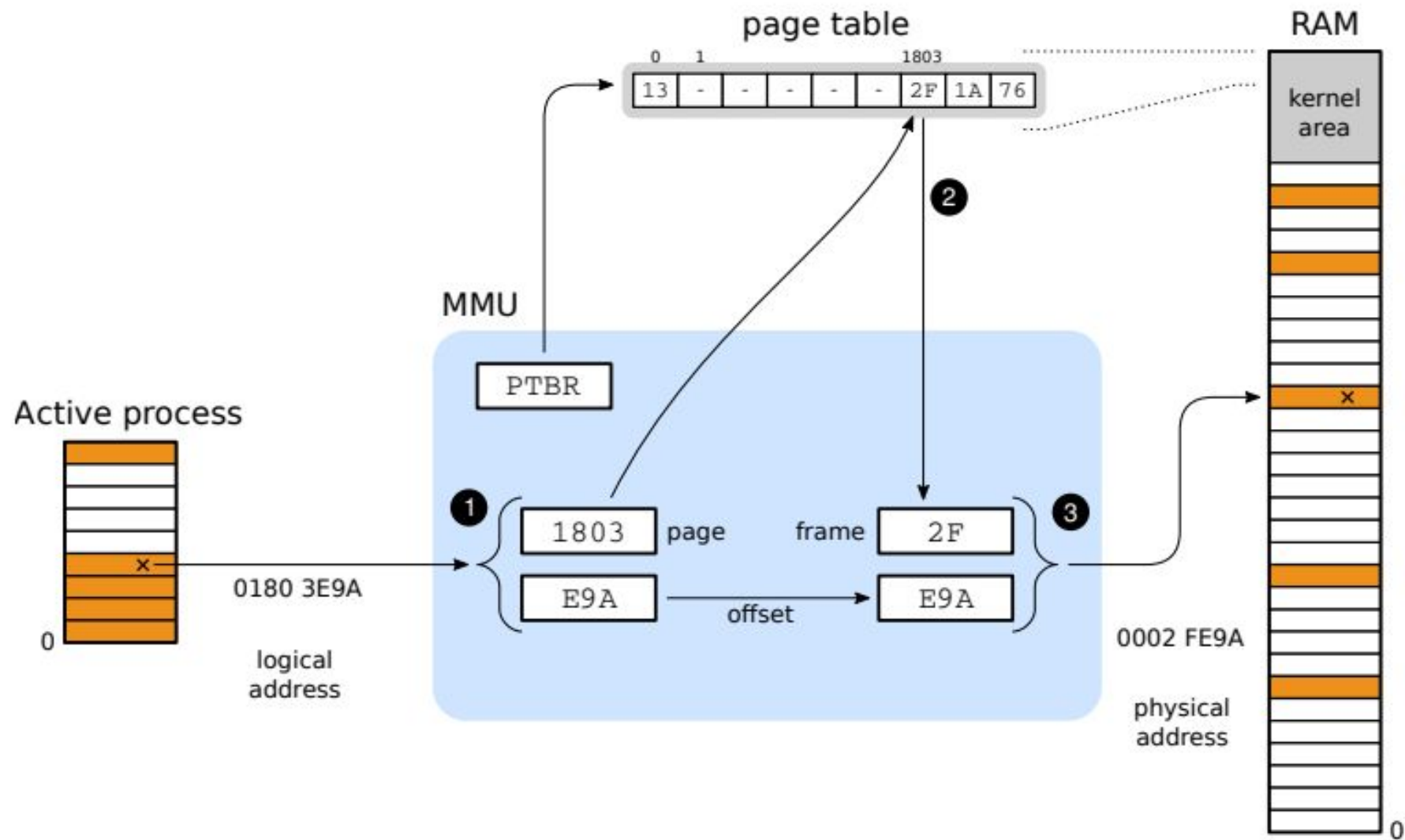


Figura 14.9: MMU com paginação.

Cada forma de organização da memória apresenta vantagens específicas:

- A organização por partições destaca-se pela simplicidade e rapidez na tradução dos endereços.
- A organização por segmentos permite que cada processo possua diferentes espaços de endereçamento, oferecendo maior flexibilidade e diferentes políticas de acesso.
- A organização por páginas disponibiliza um grande espaço de endereçamento linear e elimina a fragmentação externa.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

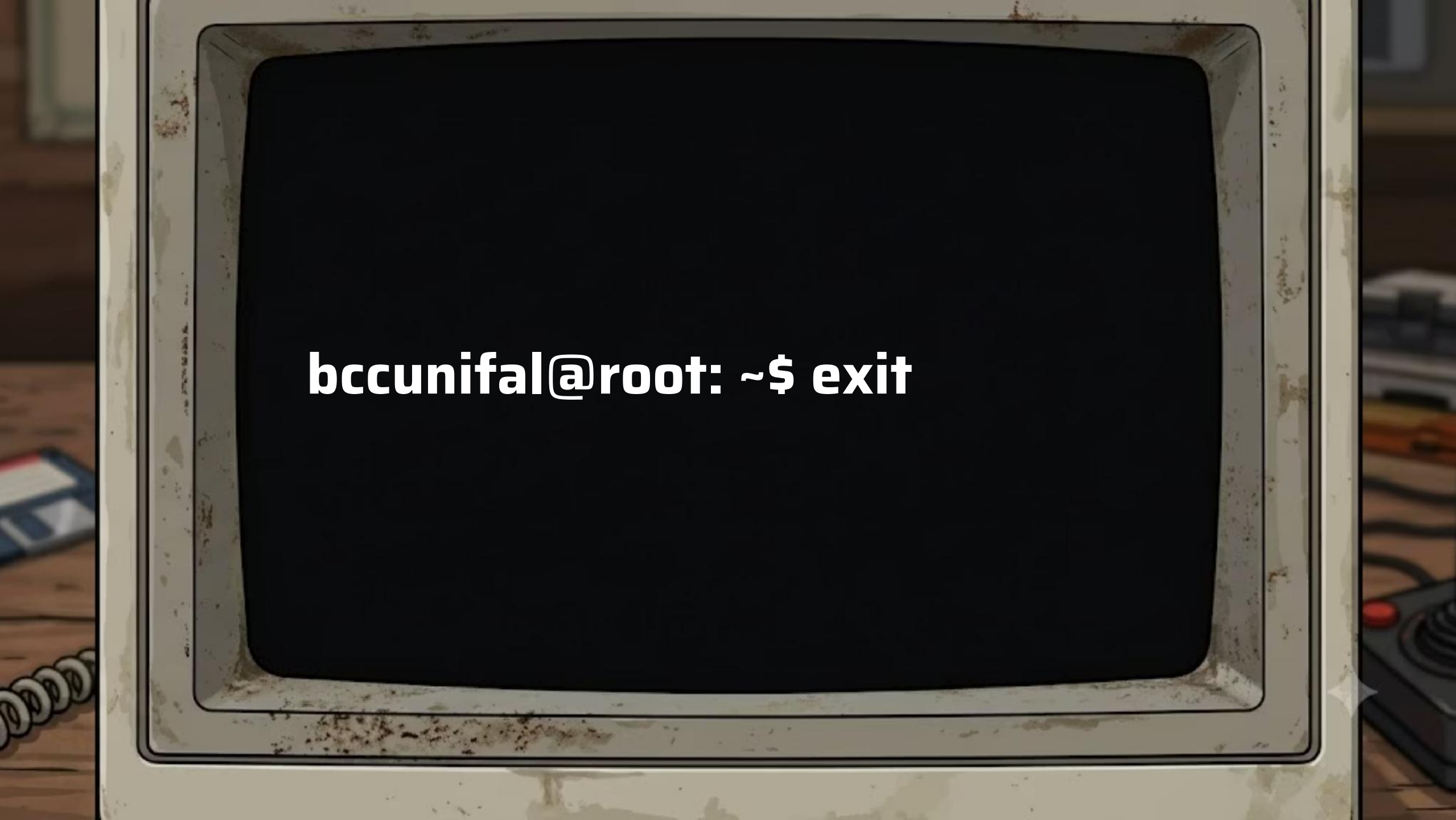
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

bccunifal@root: ~\$ exit